

# Systems Engineering Interview Preparation Guide

NOTE: You do not need to cover each and everything for the interviews. Interview process of each and every company is very different. First, look at the interview experience and job description and then at the topics for learning.

## 1. Essential Resources & References

The following links and books form the foundation of the preparation material. Prioritize the specific sections highlighted below.

| Resource                              | Type          | Focus Areas & Instructions                                                                                                                                                                              |
|---------------------------------------|---------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">Knowledgebase-SV</a>      | GitHub Repo   | OS, Networking, and General C++ Concepts. <b>Must cover:</b> C++ Performance (Return Value Optimization - RVO, Function Inlining). <b>Skip:</b> HTTP using libcurl, allocators, CRT, and the "3rd one". |
| <a href="#">Implementation-CPP</a>    | GitHub Repo   | Cover <b>all</b> topics thoroughly. High focus on standard library implementations.                                                                                                                     |
| <a href="#">System Call Deep Dive</a> | YouTube Video | Detailed explanation of system calls, context switching, and kernel interactions.                                                                                                                       |
| C++ Concurrency in Action             | Book          | Primary resource for multithreading, locks, and lock-free programming.                                                                                                                                  |

| Resource                                      | Type    | Focus Areas & Instructions                                                                                       |
|-----------------------------------------------|---------|------------------------------------------------------------------------------------------------------------------|
| <a href="https://bequant.dev">bequant.dev</a> | Website | General preparation resource for quantitative development and systems engineering. (1 month free for every user) |

## 2. Core C++ Concepts & Internals

Interviewers expect a very deep understanding of C++ beyond just syntax. You must understand how features work under the hood and their memory/performance implications.

- **Memory Management & Move Semantics:** Understand l-values, r-values, `std::move`, and how move constructors prevent unnecessary deep copies. Be prepared to explain how Smart Pointers (`std::unique_ptr`, `std::shared_ptr`, `std::weak_ptr`) handle ownership and reference counting internally.
- **Object-Oriented Internals:** How polymorphism is actually implemented. You must be able to draw and explain the **vtable** (virtual table) and **vptr** (virtual pointer), including the memory overhead per object and the performance cost of virtual dispatch.
- **Low-Level Memory Layout:** Struct padding, packing, and data alignment. Understand why compilers pad structs and how to use `#pragma pack` or `__attribute__((packed))`.
- **Casting:** Differences between `static_cast`, `dynamic_cast`, `const_cast`, and `reinterpret_cast`. Specifically, know how to use `reinterpret_cast` safely (e.g., memory-mapped hardware, serialization, casting pointer to integer types of sufficient size).
- **Modern C++ Features:** Specifically focus on **C++17 features** (e.g., structured bindings, `std::optional`, `std::variant`, `std::any`, `if constexpr`, string views).
- **Performance Enhancements:** Inline functions (and why the compiler might ignore the `inline` keyword) and Return Value Optimization (RVO / NRVO).

## 3. Data Structures & STL Implementation

Instead of just using standard library containers, interviews heavily focus on *how* they are implemented from scratch.

- **Vectors:** How dynamic arrays grow (usually doubling in size), capacity vs. size, amortized constant time complexity, and internal memory reallocation.
- **Maps & Sets (Ordered):** Implemented using self-balancing binary search trees (specifically **Red-Black Trees**). Know the properties of Red-Black Trees, how values are

sorted according to keys, and why they are chosen over AVL trees (faster insertions/deletions).

- **Unordered Maps & Sets:** Implemented using Hash Tables. Understand hash functions, collision resolution (chaining vs. open addressing), and load factors. Contrast `std::map` vs `std::unordered_map`.

## 4. Operating Systems, Linux & Tooling

Familiarity with the OS environment where systems code runs is mandatory.

- **System Calls:** Deep dive into `fork()`, `exit()`, `read()`, and `write()`. Understand what happens in the kernel when a process forks.
- **Inter-Process Communication (IPC):** How pipes work exactly (e.g., the `|` pipeline operator in Linux, unidirectional data channels, file descriptors).
- **Linux Commands & Git:** Basic familiarity, naming essential pipeline commands (`grep`, `awk`, `sed`), and version control workflows.

## 5. Concurrency & Multithreading

A core requirement for high-frequency trading and low-latency systems. Study these topics thoroughly using *C++ Concurrency in Action*.

- **Core primitives:** Threads, Mutexes, Locks (`std::lock_guard`, `std::unique_lock`), and Semaphores.
- **Condition Variables:** How to use them safely to avoid spurious wakeups.
- **Key Implementation: Bounded Buffer (Producer-Consumer Queue)**

```
#include <iostream>
#include <queue>
#include <mutex>
#include <condition_variable>

using namespace std;

template <typename T>
class BoundedBuffer {
private:
    queue<T> buffer;
    size_t capacity;
    mutex mtx;
    condition_variable not_full;
    condition_variable not_empty;
```

```

public:
    BoundedBuffer(size_t cap) : capacity(cap) {}

    void produce(T item) {
        unique_lock<mutex> lock(mtx);
        not_full.wait(lock, [this]() { return buffer.size() < capacity; });
        buffer.push(item);
        not_empty.notify_one();
    }

    T consume() {
        unique_lock<mutex> lock(mtx);
        not_empty.wait(lock, [this]() { return !buffer.empty(); });
        T item = buffer.front();
        buffer.pop();
        not_full.notify_one();
        return item;
    }
};

```

## 6. Applied DSA & System Design Topics

- **Databases:** Indexing. Know *why* we use indexing (e.g., B-Trees internally speeding up range queries and lookups) and connect it to past projects if applicable.
- **Algorithmic Coding:** Median of flowing numbers (Stream of Data). Be prepared to implement this using two Priority Queues (a max-heap for the lower half and a min-heap for the upper half).
- **System Design Execution: Order Book Implementation.** This is heavily asked in quant firms. You may be asked to design and implement an Order Book capable of fast inserts, updates, deletes, and matching limits/market orders using a combination of Hash Maps and Doubly Linked Lists or Red-Black Trees. Expect to spend up to 60 minutes on this alone.

## 7. Interview Structure & Strategy

This is a typical interview setup for Squarepoint C++ role:

- **Project/Resume Deep Dive (~10 mins):** If your projects or internships are in C++ or at a Quant firm, expect a rigorous deep dive. If your background is strictly ML/Python/Full-stack, they will lightly skim it and move straight to core systems questions. Be prepared to discuss specific technical difficulties faced during projects/internships.
- **Core Concepts Quick-fire (~7-8 mins):** Fast-paced theoretical questions (Map vs Unordered Map, Smart pointers, Virtual pointers, etc.). Keep answers concise but technically precise.
- **Deep Implementation (~60 mins):** The bulk of the interview will test your ability to design and implement complex, low-latency structures like an Order Book.